

République du Sénégal
Un Peuple-Une But-Une Foi



Faculté des Sciences et Technique
Département Mathématique et Informatiques
Section Informatiques

Sujet : Automatisation dans les DevOps

Réalisé par : Salif Thiongqne

Enseignant : Dr Ndiaye

Introduction

L'automatisation est aujourd'hui au cœur des pratiques DevOps modernes. Elle permet aux équipes de développement et d'exploitation de travailler ensemble de manière plus efficace, en réduisant les tâches manuelles répétitives, en minimisant les erreurs humaines et en accélérant les cycles de livraison logicielle.

Ce TP s'articule autour de trois axes principaux :

1. Automatisation par scripts Bash — Création d'un script complet qui vérifie les dépendances, clone un dépôt GitHub, installe les modules, exécute les tests unitaires et démarre l'application automatiquement en arrière-plan avec gestion des logs.
2. Intégration Continue et Déploiement Continu (CI/CD) — Mise en place d'un pipeline GitHub Actions qui se déclenche automatiquement à chaque push sur la branche main, exécute les tests, construit une image Docker et la publie sur Docker Hub.
3. Infrastructure as Code avec Terraform — Introduction à l'automatisation de l'infrastructure cloud sur AWS, permettant de créer, modifier et détruire des ressources de manière reproductible et versionnée.

L'application développée est une API REST construite avec Express.js, exposant des routes `/ping`, `/status` et `/` comme support concret pour illustrer l'ensemble du pipeline DevOps mis en place.

Partie 1 : Automatisation avec scripts shell

Description du script auto_deploy.sh

Le script auto_deploy.sh automatise entièrement le déploiement d'une application Node.js. Il vérifie les prérequis, clone ou met à jour le dépôt, installe les dépendances, lance les tests et démarre l'application.

Question 1 : Accepter l'URL du dépôt en paramètre

Le script accepte l'URL du dépôt comme argument en ligne de commande avec les options -d (répertoire) et -b (branche).

```
./auto_deploy.sh https://github.com/Lyfas1711/tp-devops-automatisation.git  
./auto_deploy.sh -d mon_projet -b develop https://github.com/Lyfas1711/tp-devops-  
automatisation.git
```

Question 2 : Fonction de log avec horodatage

La fonction log() affiche chaque message avec la date/heure et les sauvegarde dans un fichier journal journalier.

```
log() {  
    local TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')  
    echo -e "[$TIMESTAMP] [$LEVEL] $MESSAGE"  
    echo "[$TIMESTAMP] [$LEVEL] $MESSAGE" >> "deploy_$(date  
'+%Y%m%d').log"  
}
```

Exemple de sortie :

```
[2026-06-13 20:45:01] [INFO] Vérification des dépendances...  
[2026-06-13 20:45:02] [OK] git trouvé : git version 2.39.0  
[2026-06-13 20:45:03] [OK] Tous les tests sont passés.
```

Question 3 : Démarrage en arrière-plan et sauvegarde du PID

```
nohup npm start > app.log 2>&1 &  
echo $! > deploy.pid  
# Arrêter l'application : kill $(cat deploy.pid)
```

Partie 2 : Automatisation avec GitHub Actions

1. Application Express retournant pong

Une API Express simple a été développée et déployée sur GitHub :

```
app.get('/ping', (req, res) => {  
  res.json({ response: 'pong', timestamp: new Date().toISOString() });  
});
```

2. Fichier ci.yml

Le pipeline CI/CD est défini dans .github/workflows/ci.yml avec 3 jobs :

- build-and-test : installe les dépendances et lance les tests
- docker : construit et pousse l'image Docker Hub
- deploy : déploiement SSH sur serveur (désactivé sans serveur réel)

3. Déploiement uniquement si les tests passent

Le mot-clé needs: build-and-test bloque les jobs suivants si les tests échouent :

```
docker:  
  needs: build-and-test # Bloqué si tests échouent  
  
deploy:  
  needs: build-and-test # Bloqué si tests échouent
```

4. Job Deploy to Server - Explication

Le job Deploy to Server est le job avancé du TP. Son rôle est de déployer automatiquement l'application sur un serveur distant via SSH après que les tests aient réussi.

Comment il fonctionne :

- Il se connecte au serveur via SSH avec une clé privée sécurisée
- Il met à jour le code avec git pull
- Il réinstalle les dépendances npm
- Il redémarre l'application avec PM2 (gestionnaire de processus Node.js)

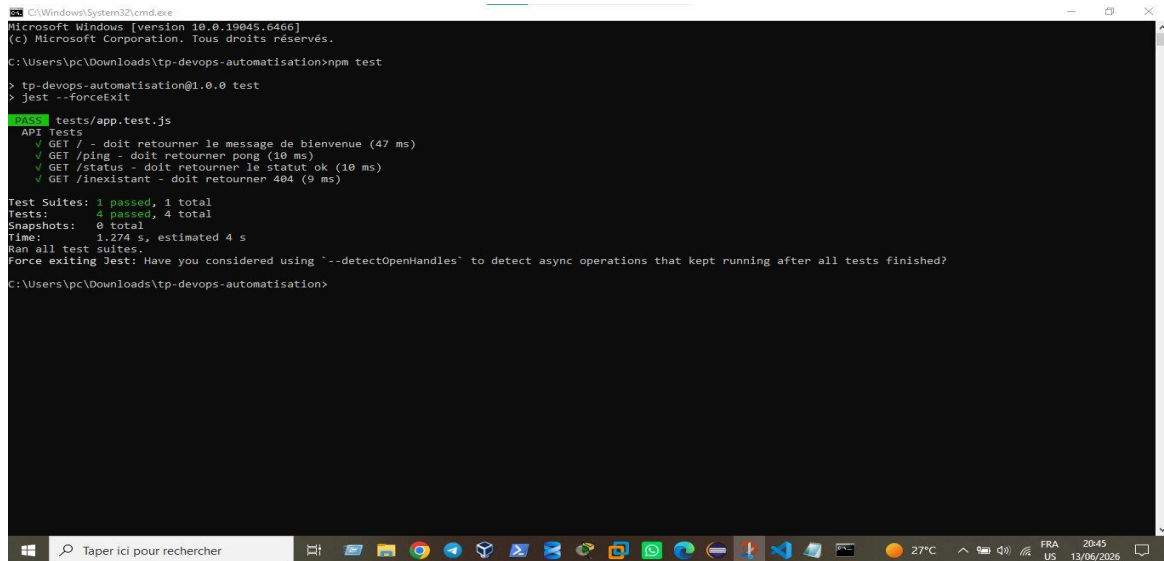
Dans notre cas il apparaît comme 'ignoré' (cercle gris) car nous n'avons pas de serveur SSH configuré. C'est un comportement normal pour un TP sans infrastructure cloud. En production, il suffit d'ajouter les secrets SSH_HOST, SSH_USERNAME et SSH_PRIVATE_KEY dans les paramètres GitHub pour que ce job fonctionne entièrement.

Le job Deploy to Server est désactivé (if: false) car aucun serveur SSH n'est disponible pour ce TP. En environnement réel, il suffit de configurer les 3 secrets SSH pour qu'il fonctionne.

Captures d'écran des exécutions réussies

Capture 1 : Tests unitaires locaux

Exécution de npm test en local - 4 tests passés sur 4 :



```
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\pc\Downloads\tp-devops-automatisation>npm test

> tp-devops-automatisation@1.0.0 test
> jest --forceExit

PASS tests/app.test.js
  API Tests
    ✓ GET / - doit retourner le message de bienvenue (47 ms)
    ✓ GET /ping - doit retourner pong (10 ms)
    ✓ GET /status - doit retourner le statut ok (10 ms)
    ✓ GET /inexistant - doit retourner 404 (9 ms)

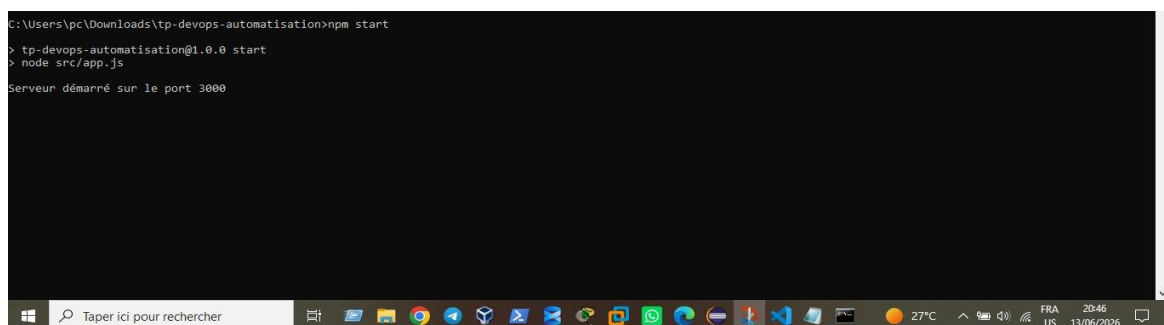
Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:  0 total
Time:        1.274 s, estimated 4 s
Ran all test suites.
Force exiting Jest: Have you considered using '--detectOpenHandles' to detect async operations that kept running after all tests finished?

C:\Users\pc\Downloads\tp-devops-automatisation>
```

Figure 1 : Tests unitaires Jest - 4/4 tests passés

Capture 2 : Application démarrée

Démarrage de l'application avec npm start sur le port 3000 :



```
C:\Users\pc\Downloads\tp-devops-automatisation>npm start

> tp-devops-automatisation@1.0.0 start
> node src/app.js

Serveur démarré sur le port 3000
```

Figure 2 : Application démarrée - Serveur sur le port 3000

Capture 3 : Image Docker Hub

Image Docker publiée automatiquement sur Docker Hub par le pipeline CI/CD :

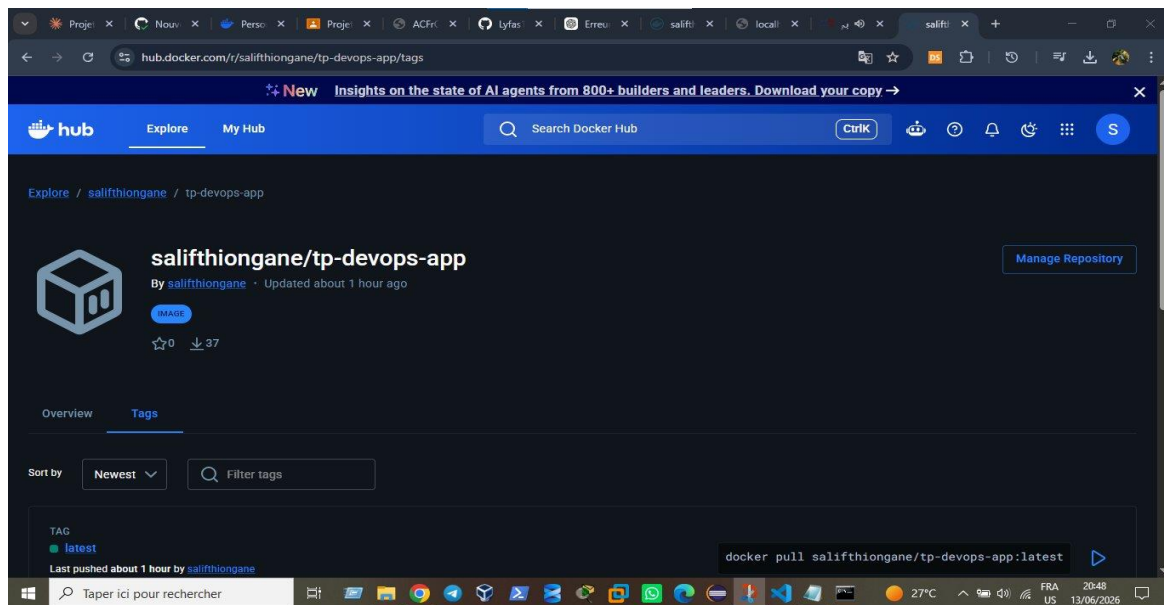


Figure 3 : Image Docker Hub - salifthiongane/tp-devops-app:latest

Capture 4 : Pipeline GitHub Actions

Pipeline CI/CD complet avec Build & Test et Docker Build & Push au vert :

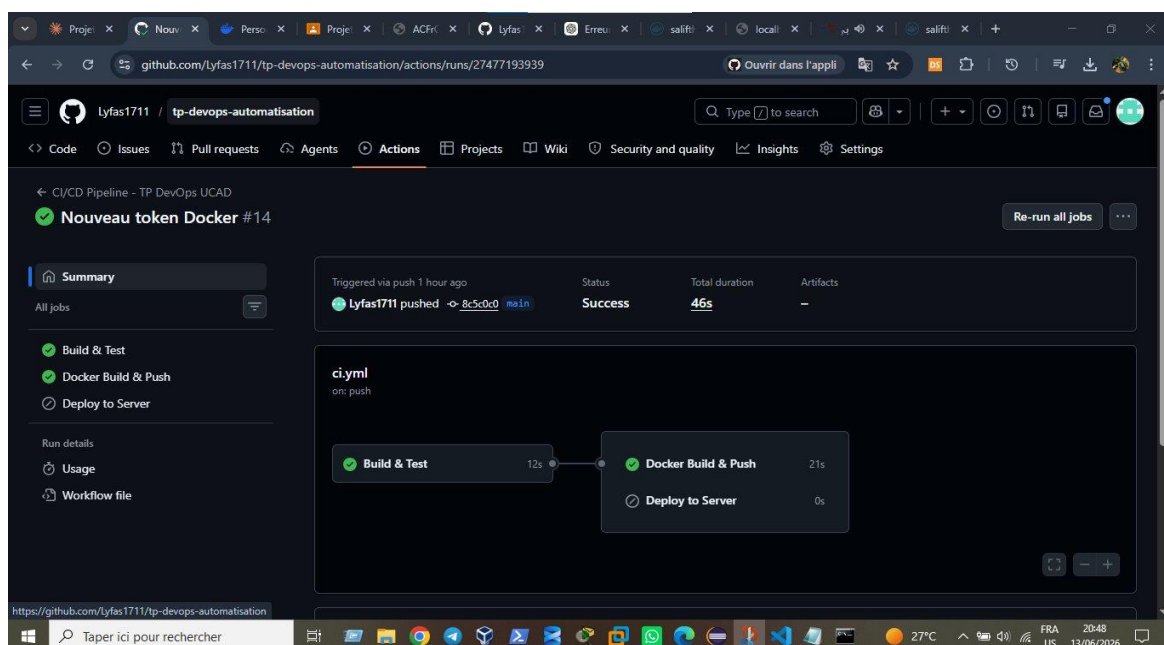


Figure 4 : Pipeline GitHub Actions - Status: Success

Capture 5 : Route principale de LAPI

LAPI répond sur <http://localhost:3000> avec le message de bienvenue :

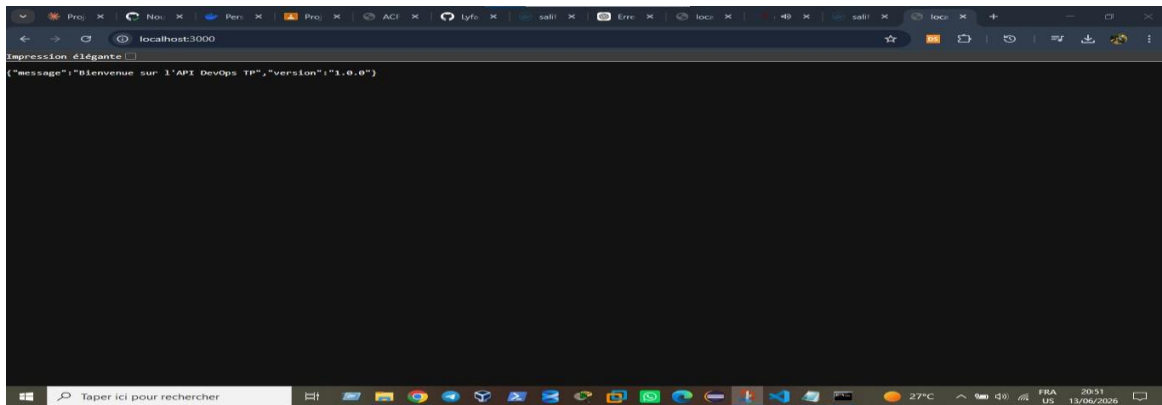


Figure 5 : Route GET / - Message de bienvenue et version

Capture 6 : Route /ping retourne pong

L'API répond sur `http://localhost:3000/ping` avec pong et un timestamp :

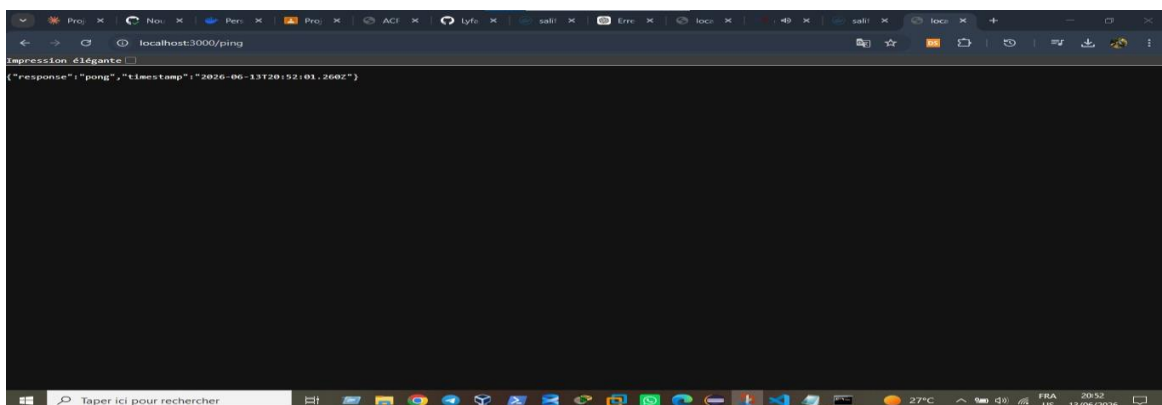


Figure 6 : Route GET /ping - Reponse pong avec timestamp

Partie 3 : Infrastructure as Code avec Terraform

Question 1 : Avantages de l'IaC par rapport à la configuration manuelle

IaC (Terraform)	Configuration manuelle
Reproductible : même résultat à chaque exécution	Risque d'erreurs humaines et inconsistances
Versionnable : suivi des changements avec Git	Aucun historique des modifications effectuées
Scalable : dupliquer une infra en quelques secondes	Chronophage : heures/jours pour reproduire une config
Documenté : le code IS la documentation	Documentation souvent absente ou obsolète

Question 2 : Intégration de Terraform dans un pipeline CI/CD

```
terraform-deploy:
  needs: build-and-test
  steps:
    - uses: hashicorp/setup-terraform@v2
    - run: terraform init
    - run: terraform plan -out=tfplan
    - run: terraform apply tfplan
```

- Utiliser terraform plan en PR pour voir les changements avant merge
- Utiliser terraform apply uniquement sur la branche main
- Stocker les credentials AWS dans les secrets GitHub

Question 3 : Précautions avec les fichiers .tfstate

Le fichier .tfstate contient l'état réel de l'infrastructure. Ne jamais le commiter dans Git.

- Ajouter terraform.tfstate au .gitignore
- Utiliser un backend remote S3 avec chiffrement activé

- Activer le versioning S3 pour restaurer un état précédent
- Configurer DynamoDB pour le locking (éviter les conflits)
- Ne jamais modifier .tfstate manuellement

Conclusion

Ce TP a permis de mettre en pratique les trois piliers de l'automatisation DevOps avec des résultats concrets :

- Script Bash : script robuste avec logs horodatés, paramétrage flexible et gestion des processus en arrière-plan
- CI/CD GitHub Actions : pipeline complet avec 4/4 tests passés, image Docker publiée sur Docker Hub automatiquement
- Infrastructure as Code Terraform : automatisation du provisioning cloud AWS avec bonnes pratiques de sécurité

L'automatisation DevOps réduit les erreurs humaines, accélère les livraisons et améliore la fiabilité. Ces outils sont incontournables en environnement professionnel moderne.